

05_INTER PROCESS COMMUNICATION

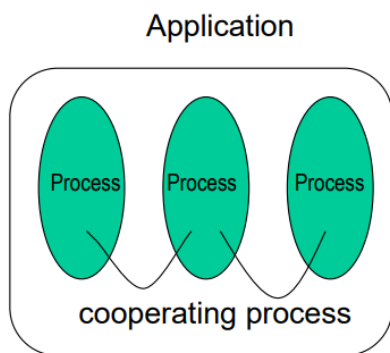
Ogni processo ha il proprio address space, e può accedere solamente a questo.
Il kernel invece può accedere a tutto.

I processi di un sistema possono essere:

- **Independent:** processi indipendenti non possono influenzare o essere influenzati dall'esecuzione di un altro processo
- **Cooperating:** processi cooperativi invece possono influenzare o essere influenzati dall'esecuzione di un altro processo

Motivi per l'uso di process cooperation:

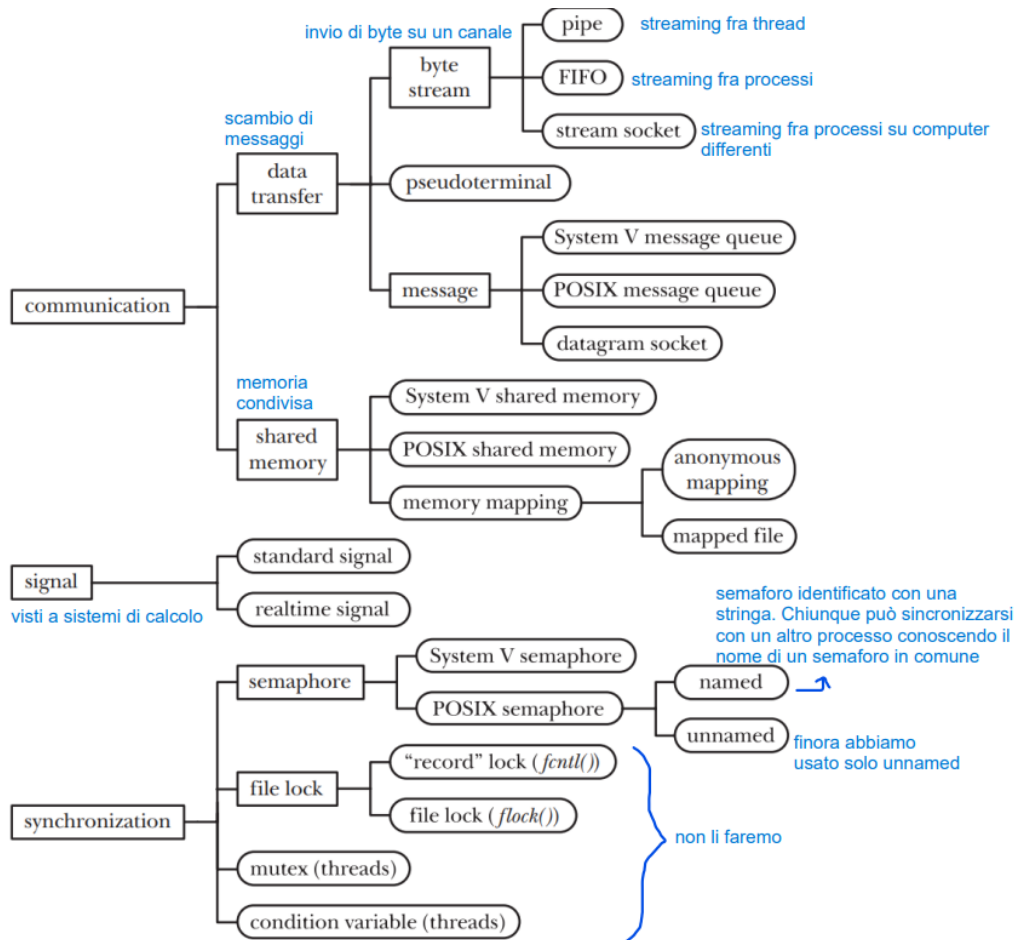
- Information sharing
- Computation speed-up
- Modularity (Modularità, le applicazioni saranno divise in moduli/sub-task)
- Convenience (Convenienza, potrebbe essere meglio lavorare con molteplici processi)



L'applicazione complessiva è progettata in modo da essere costituita da processi cooperativi

Inter-process communication (IPC): meccanismo che i processi usano per comunicare e sincronizzare le loro azioni.

Vediamo alcune soluzioni:

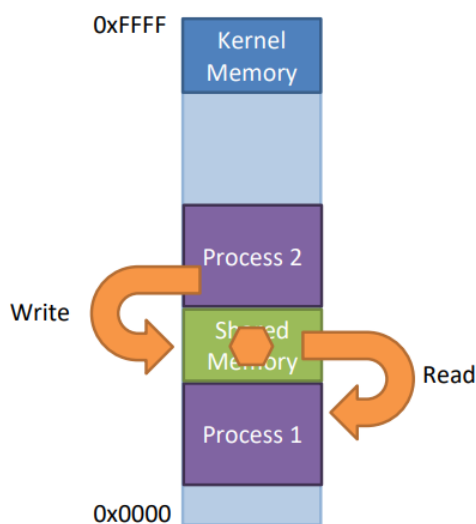


Processi cooperativi richiedono un meccanismo per interprocess communication (IPC).

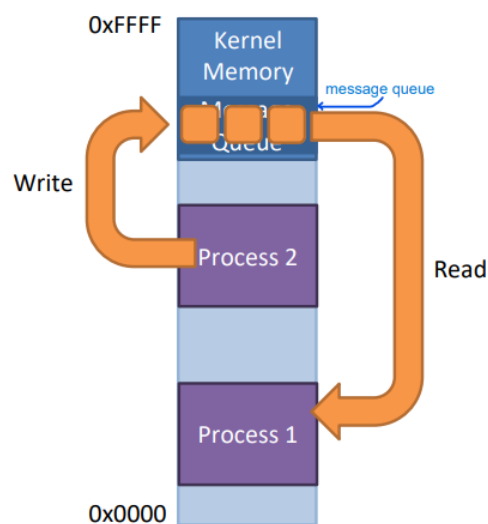
Ci sono due modelli IPC di base forniti dalla maggioranza dei sistemi:

- **Shared memory model**, i processi usano una shared memory per scambiarsi dati
- **Message passing model**, i processi si mandano messaggi fra di loro attraverso il kernel

Shared Memory



Message Passing



Con message passing ho un vantaggio in termini di concorrenza ma uno svantaggio in termini di overhead introdotto dal kernel (ogni volta devo fare passaggio al kernel).

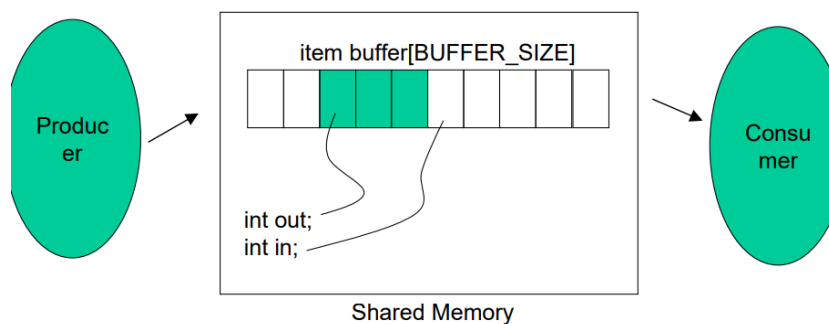
- Meccanismi di shared memory IPC:

Viene stabilita una regione di memoria condivisa fra due o più processi, attraverso l'aiuto del kernel OS (system calls).

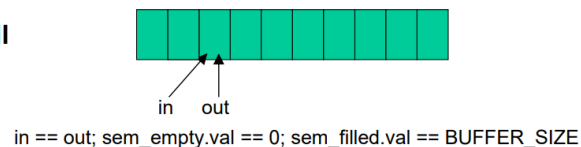
I processi possono leggere e scrivere direttamente sulla regione di memoria condivisa come un ordinario accesso in memoria (con puntatore).

Durante questa operazione il kernel non viene coinvolto, quindi è veloce.

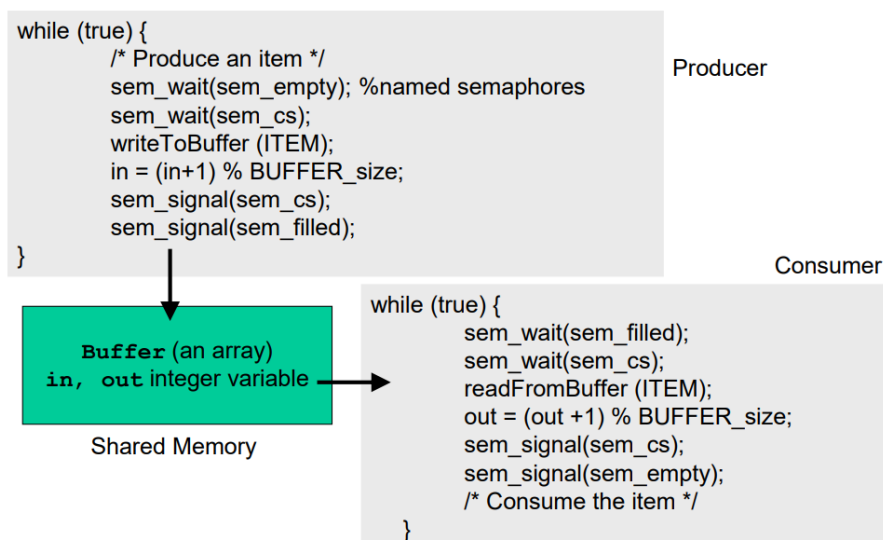
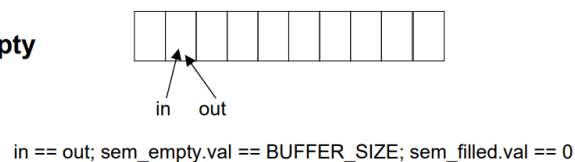
Possiamo risolvere il problema producer/consumer con buffer limitato attraverso un meccanismo di shared memory IPC:



Buffer Full



Buffer Empty



API shared memory Posix:

- **shm_open()**: create and/or open a shared memory page. Returns a file descriptor for the shared page. We can create a private shared memory (Only child processes can access the SHM too).
- **ltruncate()** o **ftruncate()**: limit the size of the shared memory page
- **mmap()**: map the memory page into the processes address space. Now you can read/write the page using a pointer
- **close()**: close a file descriptor
- **shm_unlink()**: remove a shared page. Processes with open references may still access the page

es. scrittura e lettura da shared memory

```
/* Program to write some data in shared memory */
int main() {
    const int SIZE = 4096;

    const char * name = "MY_PAGE";
    const char * msg = "Hello World!";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = (char *) mmap(0, SIZE, PROT_WRITE,
        MAP_SHARED, shm_fd, 0);
    sprintf(ptr, "%s", msg);
    close(shm_fd);
    return 0;
}

/* Program to read some data from shared memory */
int main() {
    const int SIZE = 4096;

    const char * name = "MY_PAGE";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = (char *) mmap(0, SIZE, PROT_READ,
        MAP_SHARED, shm_fd, 0);
    printf("%s\n", ptr);
    close(shm_fd);
    shm_unlink(name);
    return 0;
}
```

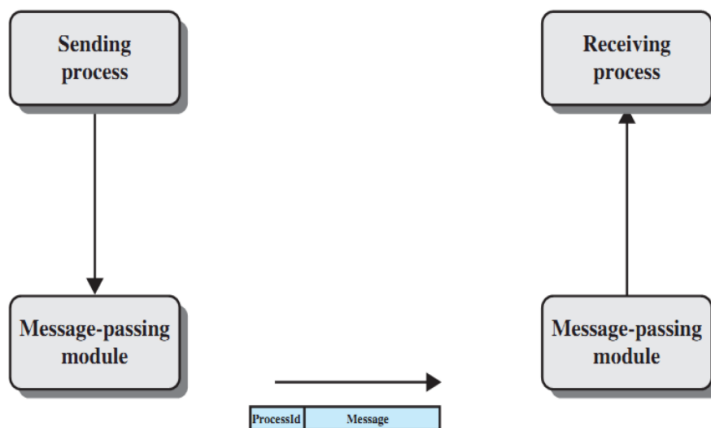
- Meccanismi di message passing IPC

Message system: processes communicate with each other without resorting to shared variables

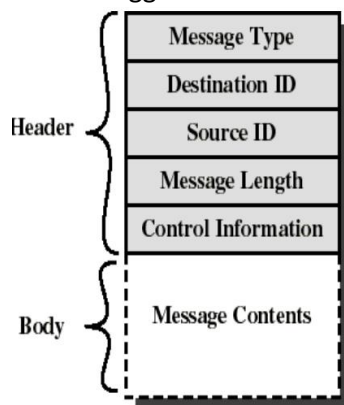
Abbiamo almeno due primitive:

- **send(destination, message)** or **send(message)**
- **receive(source, message)** or **receive(message)**

Message size is fixed or variable.



Format dei messaggi:



In Unix non ci stanno ID, solo message type.

Control information: what to do if run out of buffer space, sequence numbers, priority.

Queuing discipline: usually FIFO but can also include priorities.

Un meccanismo di message passing è **pipe (o FIFO)**, un canale di comunicazione in cui mettere u messaggi.

A pipe enables one-way communication between a parent and child. It is easy to use.

When process terminates, pipe is removed automatically.

La system call è **pipe()**.

Concetti di base su PIPE:

- permettono a più processi di comunicare come se stessero accedendo a dei file sequenziali
- il termine “pipe” significa tubo in Inglese, e la comunicazione avviene in modo monodirezionale
- una volta lette, le informazioni spariscono dalla PIPE e non possono più ripresentarsi (a meno che non vengano riscritte all'altra estremità del tubo)
- a livello di OS, le PIPE non sono altro che buffer di dimensione più o meno grande (solitamente 4096 byte). Quindi è possibile che un processo venga bloccato se tenta di scrivere su una PIPE piena.
- i processi che usano una PIPE devono essere “relazionati” come conseguenza di operazioni fork()
- le named PIPE (FIFO) permettono la comunicazione anche tra processi non relazionati
- in sistemi UNIX l'uso delle PIPE avviene attraverso la nozione di descrittore

La pipe è una astrazione file-like per inviare dati fra processi. Can be read or written to, just like a file. Permissions controlled by the creating process.

Ci sono due tipi di pipe:

- **Named pipe:** any process can attach as long as it knows the name (typically used for long lived IPC)
- **Unnamed/anonymous pipe:** only exists between a parent and its children

La comunicazione attraverso una pipe può essere full o half-duplex (can one or both ends of the pipe be read/written?).

Slide 23...

Pipe nei sistemi Unix:

<code>int pipe(int fd[2])</code>	
Descrizione	invoca la creazione di una PIPE
Argomenti	fd: puntatore ad un buffer di due interi - fd[0] : descrittore di lettura dalla PIPE - fd[1]: descrittore di scrittura sulla PIPE
Restituzione	-1 in caso di fallimento, 0 altrimenti

fd[0] è un canale aperto in lettura che consente ad un processo di leggere dati da una PIPE.
fd[1] è un canale aperto in scrittura che consente ad un processo di immettere dati sulla PIPE.
fd[0] e fd[1] possono essere usati come normali descrittori di file tramite le chiamate read() e write().

Un processo che provi a leggere da una pipe vuota rimane bloccato finché non ci sono dati disponibili.

Un processo che provi a scrivere su una pipe piena rimane bloccato finché un altro processo non ha letto (e rimosso) abbastanza dati da permettere la scrittura.

Una pipe ha una dimensione massima equivalente a 16 pagine di memoria in linux (65,536 byte in un sistema con una page size di 4096 byte).

È possibile rendere la lettura e la scrittura non bloccanti (non in questo corso).

È possibile cambiare la dimensione della pipe.

Una scrittura di $n \leq \text{PIPE_BUF}$ byte è atomica ($\text{PIPE_BUF} = 4096$ byte, si blocca se non ci sono n byte disponibili).

Una scrittura di $n > \text{PIPE_BUF}$ byte si potrebbe inframmezzare con altre `write()`, il processo rimane bloccato finché non sono stati scritti tutti gli n byte.

Nel caso di `write` non bloccanti: se $n \leq \text{PIPE_BUF}$ ma non ci sono n byte disponibili `write()` fallisce e setta `errno`; se invece $n > \text{PIPE_BUF}$ potrebbe risultare in una scrittura parziale (ma non in un errore).

Le PIPE non sono dispositivi fisici, ma logici, pertanto viene spontaneo chiedersi come un processo sia in grado di vedere la fine di un "file" su una PIPE.

Per convenzione, ciò avviene quando tutti i processi scrittori che condividevano il descrittore `fd[1]` lo hanno chiuso. In questo caso la chiamata `read()` effettuata da un lettore restituisce zero come notifica dell'evento che tutti gli scrittori hanno terminato il loro lavoro.

Allo stesso modo, un processo scrittore che tenti di scrivere sul descrittore `fd[1]` quando tutte le copie del descrittore `fd[0]` siano state chiuse (non ci sono lettori sulla PIPE), riceve il segnale `SIGPIPE`, altrimenti detto «Broken pipe».

Per fare in modo che tutto funzioni correttamente e non si verifichino situazioni di deadlock, è necessario che tutti i processi chiudano i descrittori di PIPE che non gli servono, usando una normale `close()`.

Si noti che ogni processo lettore che erediti la coppia `(fd[0], fd[1])` deve chiudere la propria copia di `fd[1]` prima di mettersi a leggere da `fd[0]` dichiarando così di non essere uno scrittore.

Se così non facesse, l'evento "tutti gli scrittori hanno terminato" non potrebbe mai avvenire se il lettore è impegnato a leggere, e si potrebbe avere un deadlock.

es. trasferimento stringhe tramite PIPE

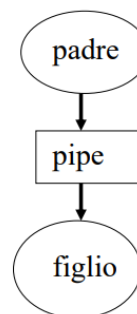
```
#include <stdio.h>
#define Errore_(x) { puts(x); exit(1); }

int main(int argc, char *argv[]) {
    char messaggio[30]; int pid, status, fd[2];

    ret = pipe(fd); /* crea una PIPE */
    if ( ret == -1 )
        Errore_("Errore nella creazione pipe");

    pid = fork(); /* crea un processo figlio */
    if ( pid == -1 ) Errore_("Errore nella fork");

    if ( pid == 0 ) { /* processo figlio: lettore */
        close(fd[1]); /* il lettore chiude fd[1] */
        while( read(fd[0], messaggio, 30) > 0 )
            printf("letto messaggio: %s", messaggio);
        close(fd[0]);
        _exit();
    }
}
```



```

/* processo padre: scrittore */
else {

    close(fd[0]);
    puts("digitare testo da trasferire (quit per
terminare):");

    do {
        fgets(messaggio, 30, stdin);
        write(fd[1], messaggio, 30);
        printf("scritto messaggio: %s", messaggio);
    } while( strcmp(messaggio, "quit\n") != 0 );

    close(fd[1]);
    wait(&status);
}
}

```

Named pipe nei sistemi Unix:

<code>int mkfifo(char *name, int mode)</code>	
Descrizione	invoca la creazione di una FIFO
Argomenti	1) *name: nome della FIFO da creare 2) mode: specifica i permessi di accesso alla FIFO
Restituzione	-1 in caso di fallimento, 0 altrimenti

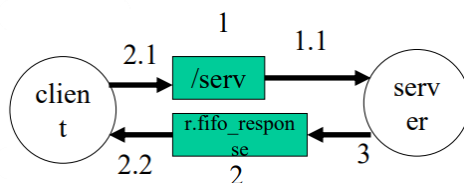
La rimozione di una FIFO dal file system avviene mediante la chiamata di sistema `unlink()`.

Normalmente, l'apertura di una FIFO è bloccante, nel senso che il processo che tenta di aprirla in lettura (scrittura) viene bloccato fino a quando un altro processo non la apre in scrittura (lettura).

Se si vuole inibire questo comportamento è possibile aggiungere il flag `O_NONBLOCK` al valore del parametro `mode` passato alla system call `open()` su di una FIFO.

Ogni FIFO deve avere sia un lettore che uno scrittore: se un processo tenta di scrivere su una FIFO che non ha un lettore esso riceve il "segnale `SIGPIPE`" da parte del sistema operativo.

es. client/server tramite FIFO



Processo server:

```
#include <stdio.h>
#include <fcntl.h>

typedef struct {
    long type;
    char fifo_response[20];
} request;

int main(int argc, char *argv[]){
    char *response = "fatto";
    int pid, fd, fdc, ret;
    request r;

    ret = mkfifo("/serv", 0666); 1
    if ( ret == -1 ) {
        printf("Errore nella chiamata mkfifo\n");
        exit(1);
    }

    fd = open("/serv",O_RDONLY);

    while(1) {
        ret = read(fd, &r, sizeof(request)); 1.1
        if (ret != 0) {
            printf("Richiesto un servizio (fifo di restituzione = %s)\n",
                r.fifo_response);
            /* switch sul tipo di servizio */
            sleep(10); /* emulazione di ritardo per il servizio */
            fdc = open(r.fifo_response,O_WRONLY);
            write(fdc, response, 20);
            close(fdc);
            3 exit(0);
        } /* end if (ret != 0) */
    }
}
```

Processo client:

```
#include <stdio.h>
#include <fcntl.h>
typedef struct { long type; char fifo_response[20]; } request;

int main(int argc, char *argv[]) {
    int pid, fd, fdc, ret;  request r;  char response[20];

    printf("Selezionare un carattere alfabetico minuscolo: ");
    scanf("%s",r.fifo_response);
    if (r.fifo_response[0] > 'z' || r.fifo_response[0] < 'a' ) {
        printf("carattere selezionato non valido, ricominciare operazione\n");
        exit(1);
    }
    r.fifo_response[1] = '\0';
    ret = mkfifo(r.fifo_response, 0666); 2
    if ( ret == -1 ) {
        printf("\n servente sovraccarico - riprovare \n");
        exit(1);
    }
}
```

```

fd = open("/serv",O_WRONLY);
if ( fd == -1 ) {
    printf("\n servizio non disponibile \n");
    ret = unlink(r.fifo_response);
    exit(1);
}

```

```

write(fd, &r, sizeof(request));
close(fd);

```

2.1

2.2

```

fdc = open(r.fifo_response,O_RDONLY);
read(fdc, response, 20);
printf("risposta = %s\n", response);

close(fdc);
unlink(r.fifo_response);
}

```

L'apertura di una pipe comporta ottenere sia il descrittore per la lettura che quello per la scrittura; bisogna poi chiudere quello che non viene usato.

L'apertura di una FIFO avviene in lettura o in scrittura.

Nella gestione si differenziano solo nella fase di apertura e chiusura.

Letture e scrittura avvengono nello stesso modo.